You've got some options. You could keep your functions in the Rails app in a directory called db/functions. From there you can manage changes but the functions won't automatically be added to your database.

Since you're working with Rails, take advantage of the fx Ruby gem (*Versioned database functions and triggers for Rails*[5]) to help you manage the lifecycle of your functions. fx provides a Rails generator that generates a SQL file where you can place your function, as well as a Migration file to add it.

You don't need to run this now, but an example of the fx function generator is:

sh/rails_fx_gem_generate_function.sh

```
bin/rails generate fx:function scrub_email
```

fx provides a create_function method as well as Active Record Migration methods to update and remove functions.

Functions are included in the application db/structure.sql file after running bin/rails db:migrate.

The fx gem is already added to Rideshare so you can explore it there.

The first time it was run it created a SQL file db/functions/scrub_email_v01.sql. Check out the db/functions directory in Rideshare to see revisions to the function.

The Migration file in Rideshare looks like this:

ruby/migration_db_function.rb

```
class CreateFunctionScrubEmail < ActiveRecord::Migration[7.0]
  def change
    create_function :scrub_email
  end
end
```

Now that you have seen the basics of creating a database function and managing the lifecycle of it in a Rails app, you're ready to return to filling out the function implementation.

Implementing the Scrub Email Function

For the scrubber function implementation you'll build on top of built-in PostgreSQL functions.

You'll use PostgreSQL SPLIT_PART()[6] function to split a string into parts.

5. https://github.com/teoljungberg/fx
6. https://www.postgresqltutorial.com/postgresql-string-functions/postgresql-split_part/